

JavaScript

Bruke JavaScript

Bjarte Wang-Kileng

HVL

25. september, 2026



**Western Norway
University of
Applied Sciences**

- ▶ Sjekk applikasjonen *Konkurranse*.
- ▶ Dette dokumentet følger utviklingen av applikasjonen.

► Versjon v1:

- Lysbilde 5 til 12: **v1k1.js**
- Lysbilde 13 til 18: **v1k2.js**
- Lysbilde 19 til 20: **v1k3.js**
- Lysbilde 21 til 25: **v1k4.js**
- Lysbilde 26 til 29: **v1k5.js**
- Lysbilde 30: **v1k6.js** og **v1k7.js**
- Lysbilde 31 til 33: **v1k8.js**
- Lysbilde 34 til 41: **v1k9.js**

► Versjon v2:

- Lysbilde 42: **v2k1.js**
- Lysbilde 43 til 44: **v2k2.js**
- Lysbilde 45 til 46: **v2k3.js**

► Versjon v3:

- Lysbilde 47: **v3k1.js**
- Lysbilde 48: **v3k2.js**
- Lysbilde 49 til 51: **v3k3.js**
- Lysbilde 52 til 53: **v3k4.js**
- Lysbilde 54 til 56: **v3k5.js**

► Versjon v4:

- Lysbilde 57 til 58: **v4k1.js**
- Lysbilde 59: **v4k2.js**
- Lysbilde 60: **v4k3.js**

JavaScript som programmeringsspråk

- ▶ JavaScript er type-svak.
 - Variabler har alltid en type, men den bestemmes ved tilordning.
- ▶ Operator *typeof* returnerer type som variabel har nå.
- ▶ Operator *instanceof* sjekker om et objekt er opprettet ved eller arver en gitt konstruktør (eller JavaScript klasse).
- ▶ Størrelse *this* avhenger av kontekst.
 - Referer ikke alltid instans av klasse selv i medlemsmetode.
- ▶ Formelle parametre trenger ikke stemme med benyttede parametre.
- ▶ Inkluderer syntaks for å definere klasser, **men er ikke klasse-basert**.
 - Klasse-konstruksjonen er dynamisk, og metoder og egenskaper kan bli lagt til, fjernet og endret under kjøring.

Bruk av *strict* modus

- ▶ Bruk alltid *strict* mode.
- ▶ Skrivefeil i variabelnavn vil gi JavaScript feil.
 - Uten *strict* blir istedet en ny variabel opprettet.
- ▶ Uten *strict* blir mange feil ignorert uten noen melding, f.eks.:
 - Forsøk på tilordning til ikke skrivbar egenskap.
 - Forsøk på å slette en ikke slettbar egenskap.
- ▶ Diverse ord kan ikke brukes som navn på variabler.
 - Bla. «eval», «arguments», «public», «private», «protected», «static» og «interface».
- ▶ Bruk av *with* konstruksjonen er ulovlig.
- ▶ Hvis metode kjøres utenfor kontekst av et objekt er «this» udefinert.

Demo med *strict* mode

- ▶ Global strict mode ved å legge «"use strict";» i starten av koden.

```
"use strict";  
  
number=7; // Error  
function minfunksjon () {  
    name="ole"; // Også dette gir Error  
}
```

- Vil gjelde for all kode lastet med samme HTML *SCRIPT* tagg.
- ▶ Noen nyere JavaScript konstruksjoner bruker alltid *strict mode*.
 - WebWorkers, bruk av *class*, og ved import/export av moduler.
- ▶ Se også utfyllende informasjon hos [Mozilla](#).

Variabel deklarasjoner – *let*

- ▶ Opprette variabel lokal til blokk:

```
"use strict";

function minfunksjon() {
  {
    let number = 77;
    console.log(number); // "number" er definert
  }
  console.log(number); // "number" er ikke definert
}

minfunksjon();
console.log(number); // "number" er ikke definert
```

- ▶ Blokk er f.eks. nærmeste løkke, if-setning, eller «`{...}`» konstruksjon.

Variabel deklarasjoner – *const*

► Opprette en konstant:

```
const PI=3.14;  
PI=4; // Feiler  
  
const a = [];  
a[0] = 77; // OK, da det er referansen som er konstant  
a = "Velkommen til DAT108"; // Feiler
```

► Konstanter med *const* har samme synlighet som *let*.

```
function minfunksjon() {  
  {  
    const number = 77;  
    console.log(number); // "number" er definert  
  }  
  console.log(number); // "number" er ikke definert  
}  
  
minfunksjon();  
console.log(number); // "number" er ikke definert
```

Variabel deklarasjoner – *var*

- Opprette variabel som er synlig i funksjon eller globalt:

```
function minfunksjon() {  
  {  
    var number = 77;  
    console.log(number); // number er definert  
  }  
  console.log(number); // number er definert  
}  
  
minfunksjon();  
console.log(number); // number er ikke definert
```

Unngå bruk av *var*

Ikke gi variabler større rekkevidde enn nødvendig.

Unngå bruk av *var*. Bruk av *var* vil gi trekk på eksamen!

Vi må likevel lære om *var* da mange eksempler på nettet bruker denne.

Hendelser **submit** og **click**

► Hendelse **click** skjer ved klikk på element.

- Brukes for å kjøre JavaScript kode.

```
function behandleklikk(clickevent) {...};  
const button = document.querySelector("button[type=button]");  
button.addEventListener("click", behandleklikk);
```

► Hendelse **submit** skjer når data i form skjema sendes til tjener.

- Sender data til webtjener og henter en side fra webtjener.

```
function behandlesubmit(submitevent) {...};  
const formElement = document.forms[0];  
formElement.addEventListener("submit", behandlesubmit);
```

Hendelse **submit**

- ▶ Triggres av:
 - Klikk på en submit knapp.
 - Bruk av *Enter* tast hvis FORM skjema har fokus.
- ▶ Nettleser har innebygd støtte for å håndtere FORM data ved **submit**:
 - Kansellerer **submit** ved ugyldige data.
 - Markere ugyldige data ved bruk av CSS-regler.
- ▶ FORM input-data og JavaScript:
 - Innebygd støtte for å arbeide med data i FORM skjema.

Submit og JavaScript

Bruk hendelse **submit** for å håndtere multiple input data.

Event objektet

- ▶ Ved hendelse opprettes et **Event** objekt for hendelsen.
- ▶ Eksempler på informasjon fra **Event** objektet:
 - HTML-element som er mål for hendelsen, egenskap.
 - Posisjon på skjermen der hendelsen skjedde (x,y posisjon).
 - Posisjon i nettleser er hendelsen skjedde (x,y posisjon).
 - Tidspunkt for hendelsen.
 - Taster og knapper som var trykket inn ved hendelsen.
- ▶ Nettleser vil gi objektet som parameter til hendeleshåndterer.

```
function behandleklikk(event) {  
    console.log(`Klikk i posisjon (${event.x},${event.y})`);  
}  
  
const button = document.querySelector("button");  
button.addEventListener("click", behandleklikk);
```

Hendelser sin metode *preventDefault*

- ▶ Noen hendelser har en standardoppførsel i nettleser. Eksempler er:
 - Klikk på en lenke (tagg *A*) vil navigere til en ny webside.
 - Hendelse **submit** vil sende data til tjener og laste en ny webside.
 - Hendelse **reset** vil nullstille et formskjema.
- ▶ Hendelser har metode *preventDefault()* for å kansellere standardoppførsel.

```
function behandlesubmit(submitevent) {  
    submitevent.preventDefault();  
}  
  
const formelement = document.forms[0];  
formelement.addEventListener("submit", behandlesubmit);
```

Strukturere koden

- ▶ Funksjoner og globale variabler.
- ▶ Klasser.
- ▶ Objekter.
- ▶ (Også andre muligheter.)

Bruk av moduler

Uansett tilnærming bør koden struktureres i JavaScript moduler.

Bruk av JavaScript moduler er ikke pensum i DAT108.

- ▶ Gir uoversikkelig kode.
- ▶ Koden blir vanskeligere å gjenbruke.
- ▶ Koden blir vanskeligere å vedlikeholde, feilsøke og skalere.
- ▶ Dårligere ytelse.
- ▶ Sikkerhetsrisiko – Skadelig kode har direkte tilgang til globale data.

- ▶ JavaScript har et *class* nøkkelord for å opprette klasser.
 - JavaScript er ikke klasse-basert, men
 - JavaScript dokumentasjonen [omtaler dette som klasser](#).
- ▶ Nøkkelord som *private*, *public* og *protected* vil gi feil.
- ▶ Nøkkelord *static* for statiske metoder og felt.
- ▶ Felt og metode er privat hvis navn starter med tegnet `#`.
- ▶ Se også [dokumentasjonen hos MDN](#).

Demo med egendefinert klasse

```
class Dice {
  #value = null;
  #maxnumber;

  constructor(maxnumber = 6) {
    this.#maxnumber = maxnumber;
  }

  getValue() {
    return this.#value;
  }

  throwDice() {
    this.#value = 1 + Math.trunc(this.#maxnumber*Math.random());
  }
}

const dice = new Dice();
dice.throwDice();

console.log(dice.getValue());
```

Samle tilhørende data og funksjoner i objekter

- Datatype **Object** brukes for å lagre data identifisert med en nøkkel.

```
const person = {  
  navn: 'Ole Olsen',  
  borndate: Temporal.PlainDate.from('2001-03-23'),  
  age() {  
    const now = Temporal.Now.plainDateISO();  
    return now.since(this.borndate, {largestUnit: "years"}).years;  
  }  
};
```

- Ikke støtte for private metoder og data.
 - Kan bruke mekanisme kalt **closure** for å skjule data og metoder.

Bruk av **Object**

Brukes for å samle variabler og funksjoner som hører sammen.

Kan også bruke **Class**, men vanligvis **Object** hvis kun en instans.

Class er en nyere konstruksjon, og har bla. private felt og metoder.

Opprette instans av **Object**

- ▶ JSON syntaks:

```
const person = {};
```

- ▶ Med bruk av *new*:

```
const person = new Object();
```

- ▶ Bruk JSON syntaks, ikke konstruksjonen med *new*.

JSON

Bruk JSON syntaks, ikke konstruksjonen med *new*.

- ▶ Legge på lytter for klikk på *BUTTON*:

```
buttonElement.addEventListener("click", funksjon);
```

- ▶ Funksjonsargumentet til *addEventListener* er en *callback*.
- ▶ En *callback* er en funksjon gitt som parameter i et funksjonskall.
- ▶ Observer at det er selve funksjonen *funksjon* som argumentet, **ikke** resultatet av å kjøre *funksjon*.

Callbacks og funksjonskall

- ▶ Argument er funksjonen selv.
- ▶ Argumentet er **ikke** resultatet av å kjøre callback funksjonen.
- ▶ Der er normalt ingen parenteser «()» bak en callback parameter:
 - Riktig:

```
buttonElement.addEventListener("click", funksjon);
```

- Feil:

```
buttonElement.addEventListener("click", funksjon());
```

Callbacks og størrelsen *this*

- ▶ I hendelseshåndterer er *this* HTML elementet for hendelsen.

```
function eventhandler(){  
    console.log(`'this' vil være HTML elementet: ${this}`);  
}  
  
button.addEventListener("click",eventhandler);
```

- ▶ Gjelder også om hendelseshåndterer er en medlemsmetode til et objekt eller klasse.

Problematiske verdi av *this*

Når hendelseshåndterer er en medlemsmetode til et objekt eller en klasse burde *this* være objektet eller instansen.

Istedet vil *this* være HTML elementet som er mål for hendelsen.

Bruk *bind* for å angi verdi til *this*

- ▶ Funksjoner har metode *bind* for å angi verdi til *this*.

```
const nyfunksjon = gammelfunksjon.bind(nythis);
```

- ▶ Kodens over kjører ikke *gammelfunksjon*, men lager en ny funksjon som kan kjøres senere:

```
nyfunksjon();
```

- ▶ Kodens under vil kjøre *gammelfunksjon* med angitt verdi av *this*:

```
gammelfunksjon.bind(nythis)();
```

Bruk *bind* for å løse problemet med *this*

- Når **click** trigger *metode()* vil *this* være en instans av klassen.

```
class MinKlasse {  
  controller (root) {  
    const button = root.querySelector("button");  
    button.addEventListener("click",this.metode.bind(this));  
  }  
  
  metode(event) { ... }  
}  
  
const instans = new MinKlasse(document.getElementById("rootid"));
```

Navngitte funksjoner og anonyme funksjoner

- ▶ Eksempel med navngitt funksjon:

```
function y(x) {return 2*x}
```

- ▶ Anonym funksjon og tilordning til variabel:

```
const y = function (x) {return 2*x};
```

- ▶ Anonym funksjon som argument til annen funksjon:

```
g("A", 32, function (x) {return 2*x});
```

Pil-notasjon for anonym funksjon

- ▶ Pil-notasjon og tilordning til variabel:

```
const y = (x) => {return 2*x};
```

- ▶ Pil-notasjon for funksjon som argument til annen funksjon:

```
// Anonym funksjon som argument til funksjon  
g("A", 32, (x) => {return 2*x});
```

Pil-notasjon for funksjoner

- ▶ Kan utelate parenteser hvis kun ett argument:

```
(x) => {return 2*x};  
  
// Ekvivalent uttrykk  
  
x => {return 2*x};
```

- ▶ Kan utelate nøkkelordet return og krøllparenteser rundt en funksjonskropp som kun returnerer en verdi:

```
x => {return 2*x};  
  
// Ekvivalent uttrykk  
  
x => 2*x;
```

- ▶ Les mer om [Arrow functions](#) i Mozilla sin dokumentasjon.

Bruk pil-notasjon for å løse problemet med *this*

- ▶ Den mest moderne tilnærming til problemet.
- ▶ Når **click** trigger *metode()* vil *this* være en instans av klassen.

```
class MinKlasse {  
  controller (root) {  
    const button = root.querySelector("button");  
    button.addEventListener("click", event => this.metode(event));  
  }  
  
  metode(event) { ... }  
}  
  
const instans = new MinKlasse(document.getElementById("rootid"));
```

Ulikheter mellom pil-notasjon og *function* for funksjon

- ▶ Pil-notasjon funksjon lager ikke sin egen binding av **this**.
 - Bra for å løse problemet med verdien på *this*.
- ▶ Pil-notasjon kan ikke brukes for klasse- eller objekt-metoder.
- ▶ Pil-notasjon for funksjon gir ingen binding av objektet **arguments**.
 - Objektet **arguments** er ikke pensum.
- ▶ Pil-notasjon funksjon har ikke en egenskap **prototype**.
 - Funksjoner sitt objekt **prototype** er ikke pensum.

Event sin egenskap *target*

- ▶ **Event** sin egenskap *target* er HTML elementet som er mål for hendelsen.

```
function behandleklikk(event) {  
    const tagg = event.target.tagName;  
    console.log(`Klikk på HTML element med tagg ${tagg}`);  
}  
  
document.body.addEventListener("click", behandleklikk);
```

- ▶ HTML element som er mål for hendelsen trenger ikke være elementet som kjører hendelsehåndterer.

- ▶ Klasse for å arbeide med FORM data.
- ▶ Oppretter nøkkel/verdi par for FORM sine input felter.
 - Nøkkel vil være feltet sitt *name* attributt.
- ▶ En **FormData** instans lar oss for eksempel:
 - Hente ut input data.
 - Iterere gjennom alle input data.
 - Legge til og slette input data.
 - Endre input data.
 - Sjekke om data finnes.

- ▶ En mengde JavaScript liste-type objekter inkluderer [iteratorer](#).
- ▶ Iterator lar oss gjennomløpe liste-elementene med en `for...of` løkke:

```
const formData = new FormData(event.target);
for (const pair of formData) {
  const key = pair[0];
  const value = pair[1];

  // Her følger kode som bruker key og value
}
```

- ▶ Et nøkkel/verdi par blir returnert som en **Array** med to elementer:
 - Indeks 0 er nøkkel til liste-elementet.
 - Indeks 1 er verdi til liste-elementet.

Arbeide med egenskap til objekt

► JSON:

```
person = {"firstname": "Ole"};
```

► JSON også uten hermetegn på egenskap:

```
person = {firstname: "Ole"};
```

► Notasjon med punktum:

```
person.firstname = "Ole";
```

► Bruk av [...]:

```
person["firstname"] = "Ole";
```

► Kan også gjøres med statiske metoder til **Object**.

- **Object** sine statiske metoder er ikke pensum.

Flere syntakser for å opprette objekt

► Via variabelverdier:

```
const number = 121;  
const name = "Ole";  
const person = { "id": number, "navn": name };
```

► Via variabler:

```
const id = 121;  
const navn = "Ole";  
const person = { id, navn }; // {"id": id, "navn": navn };
```

► Ved uttrykk for egenskap:

```
const p1 = "id";  
const p2 = "navnet";  
const person = { [p1]:121, [p2.substring(0,4)]: "Ole" };  
// Konstanten person blir da {"id": 121, "navn": "Ole" }
```

Pakke ut fra objekt til variabler

► Pakke ut fra objekt:

```
const person = { "id": 121, "navn": "Ole" };  
const {id,navn} = person;
```

► Gi navn til variablene når objekt pakkes ut:

```
let {id:studentnummer, navn:fornavn} = person;  
// Gir studentnummer som person.id og  
// fornavn som person.navn
```

Rest og spread operatoren

- Spread operator lager parameterliste fra innhold i tabell.

```
const tabell=["a", "b", "c"];  
  
f(...tabell); // Gir følgende kall f("a", "b", "c")
```

- Rest operator lager forekomst av **Array** fra parameterliste:

```
function f(...tabell) {  
    ...  
}  
  
f("a","b"); // Oppretter tabell som ["a", "b"]
```

- Operatorene kan benyttes på alle liste-type objekter med iterasjoner.

```
const formdata = new FormData(event.target);  
sjekkformdata(...formdata);
```

Pakke ut fra tabell til variabler

► Pakke ut tabell:

```
[a,b] = [2,5]; // Variabel "a" settes lik 2, og "b" til 5
```

► Bytte om av to variabler:

```
[a,b] = [b,a]; // Variabler "a" og "b" bytter verdier og typer
```

► Pakke ut fra tabell til variabler, og samtidig redefinere tabell:

```
[a,b,...tabell] = tabell;
```

► Ignorere noen verdiene fra tabell:

```
[a,,c] = [1,2,3,4,5]; // Variabel "a" settes lik 1 og "c" til 3
```

► Fjerne de to første verdier i tabell:

```
[,,...tabell] = tabell;
```

Funksjoner og pakke ut fra tabell

- Pakke ut fra tabell til formelle parametre:

```
function summer([a,b]) {return a+b};  
const tall = [2,5];  
const result = summer(tall);
```

- Standardverdier ved utpakking av tabell:

```
function produkt([a,b=1,c=2]) {return a*b*c};  
const tall = [12,3];  
const result = produkt(tall); // c vil få verdi fra default 2
```

- Pakke ut fra tabell som returverdi fra funksjon:

```
function numbers() {return [1,7,4]}  
const [a,b,c] = numbers(); // a lik 1, b lik 7 og c lik 4
```

Noen ECMA primitive datatyper

- ▶ **Undefined**
- ▶ **Null**
 - Se også `typeof null`.
- ▶ **Boolean**
- ▶ **Number**
- ▶ **String**

Mer om datatyper

- ▶ **Boolean**, **Number** og **String** har en tilsvarende wrapper-klasse.
- ▶ Bruk av wrapper-klassene er unødvendig og gir misforståelser.
- ▶ Ikke:

```
const navn = new String("Ole");
```

a

- ▶ Bedre:

```
const navn = "Ole";
```

- ▶ Alle metoder og egenskaper fra wrapper-klassene er tilgjengelige også for de primitive typene **Boolean**, **Number** og **String**.

Legge til HTML elmenter fra JavaScript

- ▶ Se også forelesning om DOM, side 32.
- ▶ Noen HTML-elementer har i JavaScript egne metoder for å operere på DOM strukturen.
- ▶ [HTMLTableElement](#) (tagg `<table>`) og [HTMLTableSectionElement](#) (tagger `<tbody>`, `<thead>` og `<tfoot>`) har metode [insertRow\(\)](#) for å legge til ny tabellrekke.
 - Med parameter `-1`, legg til som siste rekke.
- ▶ [HTMLTableRowElement](#) (tagg `<tr>`) har metode [insertCell\(\)](#) for å legge til en tabellcelle (tagg `<td>`).
 - Med parameter `-1`, legg til som siste tabellcelle i rekke.

Mer om JavaScript Array

- ▶ Opprette tabell ved bruk av konstruktørfunksjonen **Array**:

```
const farger = new Array("rød", "grønn", "blå", "gul");
```

eller

```
const farger = Array("rød", "grønn", "blå", "gul");
```

- ▶ Opprette tabell ved bruk av JSON syntaks:

```
const farger = ["rød", "grønn", "blå", "gul"];
```

Arbeide med Array

- ▶ Liste-type objekt, og nokså ulikt tabeller i Java.
- ▶ Antall elementer kan endres under kjøring.
- ▶ Kan blande elementer av forskjellige typer i samme tabell.
- ▶ Bruk av *new* med **Array** er valgfritt.
- ▶ Kan opprette en tom tabell ved å gi kun lengden som argument:

```
const farger = new Array(100);
```

Bruk JSON for Array

Bruk JSON syntaks, som er entydig og lettere å bruke.

Noen **Array** metoder

- ▶ Metode `forEach()` utfører en funksjon på alle elementene i tabell.

```
const farger = ["rød", "grønn", "blå"];

farger.forEach(
  farge => { console.log(farge) }
);
```

- ▶ Metode `findIndex()` gis en testfunksjon som argument og returnerer indeks til første element der testfunksjonen er sann.

```
const indeks = farger.findIndex(farge => farge==="grønn");
```

- Hvis testfunksjonen er usann for alle elementer returneres verdien **-1**.
- ▶ Statisk metode `from` lager en **Array** fra en liste-type objekt.
 - Kan da bruke **Array** sine metoder på listen.

Array.from(), findIndex() og insertRow() kombinert

- ▶ **Array** metode *findIndex()* returnerer **-1** hvis element ikke finnes.
- ▶ *insertRow()* legger på slutten av HTML tabell hvis argument er **-1**.
- ▶ Ny rekke i HTML tabell som er sortert stigende på deltagernummer:
 - HTML kode:

```
<table>
  <tr><td>9</td><td>Hanne Hannesen</td></tr>
</table>
```

- JavaScript:

```
const table = document.querySelector("table");
const HTMLTableRowsAsArray = Array.from(table.rows);
const nydeltager = { startnummer: 127, navn: "Anne Annesen" };

const indeks = HTMLTableRowsAsArray.findIndex(
  rekke => {
    const startnummer = Number(rekke.cells[0].textContent);
    return nydeltager.startnummer < startnummer;
  });
const newRow = table.insertRow(indeks);

// Må videre legge inn tabellceller i den nye rekken for Anne Annesen
```

FORM input validering med HTML attributter

- ▶ Validering kan gjøres med HTML attributter uten bruk av JavaScript.

```
<input name="navn" type="text" required ...  
      pattern="\s*\p{Lu}\p{Ll}+((\s+|-)\p{Lu}\p{Ll}+)*\s*>
```

- ▶ Nettleser vil ignorere hendelse **submit** fra FORM hvis ugyldige data.
- ▶ CSS pseudo-klasser **:valid** og **:invalid** kan markere gyldige og ugyldige data.
 - Vi trenger normalt kun **:invalid**.

- ▶ Egenskap *dataset* gir tilgang til “*data-**” attributter.
 - Gi verdi til HTML attributt *data-startnummer* med JavaScript kode:

```
const rekke = document.querySelector("tr");  
rekke.dataset.startnummer = 27;
```

- Se også side 40 i DOM forelesning.

- ▶ Bruke *querySelector()* for å finne element ut fra verdi på attributt:

```
const rekke = document.querySelector('tr[data-startnummer="27"]');
```

- ▶ Finne første ugyldig (tilstand *invalid*) FORM kontroll-element:

```
const inputelement = document.forms[0].querySelector(':invalid');
```

Bolske operatører og likhet

- ▶ Ulike datatyper i JavaScript kan lagre samme verdi:

```
const student1 = "Per";           // Datatype "String"  
const student2 = String("Gro");   // Datatype "String"  
const student3 = new String("Ole"); // Datatype "Object"
```

- ▶ Operatører som sjekker både type og verdi:

===: Lik type og verdi

!==: Ulik type og/eller ulik verdi

- ▶ Operatører som sjekker kun verdi:

==: Lik verdi, uansett type

!=: Ulik verdi

- ▶ Aldri bruk *new* med **Boolean**, **Number** og **String**.

- Bruk av wrapper-klassene er unødvendig og gir misforståelser.

- ▶ Bruk operatører som sjekker både type og verdi.

Set for unike verdier

- ▶ Brukes for å lagre unike verdier av hvilken som helst type.
- ▶ Hvis verdi allerede eksisterer i **Set** skjer ingenting.

```
const startnummre = new Set();
startnummre.add(27);
startnummer.add(27);    // Blir ignorert da 27 === 27
startnummer.add("27");  // Blir lagt til da 27 !== "27"
```

- ▶ Kan sjekke om verdi allerede finnes i **Set**:

```
if (startnummre.has(27)) {
    console.log("Startnummer 27 finnes allerede!");
}
```

Set og *forEach()*

- ▶ Et **Set** kan gjennomløpes med *forEach()*:

```
startnummre.forEach(  
  nummer => { console.log(`Deltager med startnummer ${nummer}`) }  
);
```

- ▶ Rekkefølge er gitt av innsettingsrekkefølge.
 - Ikke mulig å sette inn et element tidligere i rekkefølgen.

Map for unike verdier

- ▶ Brukes for å lagre nøkkel-verdi par.
 - Nøkkel vil være unik i en **Map**.
- ▶ Nøkkel kan være av hvilken som helst type.
- ▶ Opprette:

```
const deltagere = new Map();
deltagere.set(127, {"startnummer":127, "navn": "Ole"});

// Oppdatere "navn" på deltager 127
deltagere.set(127, {"startnummer":127, "navn": "Per"});

// Legger til ny deltager "127" da "127" !== 127
deltagere.set("127", {"startnummer":127, "navn": "Per"});
```

Map og *forEach()*

- ▶ En **Map** kan gjennomløpes med *forEach()*:

```
function visdeltager(value, key) {  
  console.log(`Deltager ${key} er ${value.navn}`);  
}  
deltagere.forEach(visdeltager);
```

- ▶ Rekkefølge er gitt av innsettingsrekkefølge.
 - Ikke mulig å sette inn et element tidligere i rekkefølgen.

Array for å speile en HTML tabell

- ▶ I noen situasjoner er det nyttig å speile en HTML tabell i en Array.
 - F.eks. hvis et filter skal fjerne noen av rekkene fra visning.
- ▶ I motsetning til **Map** og **Set** kan rekkefølge ved *forEach()* endres.
 - Nye rekker kan settes inn foran tidligere.
 - Rekker kan flyttes på.
- ▶ Inkludere mange nyttige metoder for å arbeide med tabellelementene.
- ▶ Metodene *Array.from()* og **rest operator** kan lage en **Array** fra andre typer lister.

Array for unike verdier

- ▶ **Array** kan lagre samme verdi flere ganger.
- ▶ Kan sjekke om element finnes fra før med f.eks. `find()` og `findIndex()`.

```
const deltagere = [  
  {'startnummer': 9, 'navn': 'Hanne Hannelsen'},  
  {'startnummer': 57, 'navn': 'Gro Grosen'}  
];  
  
const deltagerfratabell = deltagere.find(  
  deltager => deltager.startnummer === 27  
);  
if (deltagerfratabell === undefined) {  
  console.log("Deltager 27 finnes ikke");  
}
```

- ▶ Et eksempel med `findIndex()` har blitt demonstrert tidligere.

Legge til og fjerne fra start og slutt av **Array**

- ▶ Metode *unshift()* setter inn elementer på start av tabell.

```
const deltagere = [  
  { startnummer: 127, navn: "Anne Annesen" },  
  { startnummer: 567, navn: "Per Persen" }  
];  
  
const nydeltager = { startnummer: 65, navn: "Jo Josen" };  
const nylengde = deltagere.unshift(nydeltager);
```

- ▶ Metode *push()* setter inn elementer på start av tabell.

```
const nylengde = deltagere.push(nydeltager);
```

- ▶ Metode *shift()* henter ut og fjerner første element fra tabell.

```
const fjernetdeltager = deltagere.shift();
```

- ▶ Metode *pop()* henter ut og fjerner siste element fra tabell.

```
const fjernetdeltager = deltagere.pop();
```

HTMLFormElement sin egenskap *elements*

- ▶ **Document** egenskap **forms** er en liste av alle FORM elementer.
 - Indeksert både på posisjon $[0, n-1]$ og attributter *id* og *name*.
- ▶ **HTMLFormElement** egenskap **elements** er en liste av alle FORM kontrollelementer.
 - Indeksert både på posisjon $[0, n-1]$ og attributter *id* og *name*.
 - FORM kontrollelementer er bla. *INPUT*, *SELECT* og *TEXTAREA*.

- ▶ Hendelse **input** trigger når bruker endrer innhold i et HTML *INPUT*, *SELECT* eller *TEXTAREA* element.

```
const startnummerInput = document.forms[0].elements["startnummer"];

startnummerInput.addEventListener(
  'input',
  event => console.log(event.target.valueAsNumber);
);
```

Sette gyldighet av FORM data med JavaScript

- ▶ Kontrollelement metode *setCustomValidity()* for egen feilmelding.

```
const target = document.forms[0].elements["startnummer"];  
  
target.setCustomValidity("Startnummer er i bruk");  
target.title = "Startnummer er i bruk"; // For Chromium nettlesere
```

- ▶ Tilstand vil være **invalid** og feilmeldingen vises ved mus på element.
 - Nettleser vil kansellere hendelse **submit**.
- ▶ Tilstand settes til **valid** ved å fjerne feilmelding.

```
target.setCustomValidity("");  
target.title = ""; // For Chromium baserte nettlesere
```

- ▶ Et element kan være **invalid** også av andre grunner.
 - Egen feilmelding opphever ikke attributter som *required* og *pattern*.

Bestemme FORM kontroll element sin tilstand

- ▶ FORM kontroll element egenskap *validity* angir gyldighet for elementet.
- ▶ Bolsk egenskap *valid* angir om elementet har tilstand **valid**.

```
if (target.validity.valid) {  
    console.log("Alt OK");  
}
```

- ▶ Hvis **invalid**, kan finne årsak.
 - HTML:

```
<INPUT required pattern="..." ...>
```

- JavaScript:

```
if (target.validity.patternMismatch) {  
    console.log("Verdi stemmer ikke med HTML attributt pattern");  
} else if (target.validity.valueMissing) {  
    console.log("HTML attributt required, men verdi mangler");  
} else if (target.validity.customError) {  
    console.log("Ikke-tom tekst satt med setCustomValidity()");  
}
```

Ordne rekkefølge på **Array** med *sort()*

- ▶ Muterende metode *sort()* sorterer tabell.
- ▶ Kan ta ett argument, en funksjon som gis to elementer fra tabellen.

```
const deltagere = [  
  { startnummer: 567, navn: "Per Persen" },  
  { startnummer: 127, navn: "Anne Annesen" }  
];  
  
function sammenlign(deltagerA,deltagerB) { ... }  
deltagere.sort(sammenlign);
```

Funksjonen *sammenlign(A,B)* må returnere et tall:

- Hvis negativt skal *A* være før *B* i tabellen.
- Hvis positivt skal *A* være etter *B* i tabellen.
- Hvis 0 eller **NaN** oppfattes *A* og *B* som like.

- ▶ Sortere tabellen **deltagere** stigende på deltagernummer:

```
deltagere.sort((A,B) => A.startnummer - B.startnummer);
```

- ▶ Ikke muterende metode *toSorted()* som *sort()*, men lager ny tabell.

Ordne rekkefølge på **Array** med *splice()* og *findIndex()*

- ▶ Muterende metode *splice()* kan modifisere tabell.
- ▶ Metoden tar ett eller flere argumenter:
 - Første argument: Posisjon i tabellen
 - Andre argument: Antall elementer som skal fjernes
 - Resten av argumentene: Elementer som skal settes inn i tabellen.
- ▶ Sette inn ny deltager i riktig posisjon i sortert tabell:

```
const deltagere = [  
  { startnummer: 127, navn: "Anne Annesen" },  
  { startnummer: 567, navn: "Per Persen" }  
];  
  
const nydeltager = { startnummer: 65, navn: "Jo Josen" };  
const posisjon = deltagere.findIndex(  
  deltager => nydeltager.startnummer < deltager.startnummer  
);  
  
if (posisjon === -1) deltagere.push(nydeltager);  
else deltagere.splice(posisjon, 0, nydeltager);
```

- ▶ Ikke muterende metode *toSpliced()* som *splice()*, men lager ny tabell.

- ▶ Legge inn element riktig i en sortert tabell er en $O(n)$ operasjon.
 - Bruk *splice()* og *findIndex()*.
- ▶ Sortere hele tabellen er en $O(n \log(n))$ operasjon.
 - Primitiv sortering er $O(n^2)$.

Sortering og eksamen

En løsning som sorterer tabell på nytt for hvert nytt element vil gi trekk på eksamen.

Array metode *map()*

- ▶ Metode *map()* tar en funksjon som argument.
- ▶ *map()* returnerer tabell der funksjonen er kjørt på alle elementene.

```
const deltagere = [
  {"startnummer": 9, "navn": "Hanne Hannesen"},
  {"startnummer": 57, "navn": "Gro Grosen"}
];

const deltagereHTMLTekst = this.#deltagere.map(
  d => `<tr><td>${d.startnummer}</td><td>${d.navn}</td></tr>`
);
```

- ▶ Koden over vil produsere følgende tabell:

```
const deltagereHTMLTekst = [
  "<tr><td>9</td><td>Hanne Hannesen</td></tr>",
  "<tr><td>57</td><td>Gro Grosen</td></tr>"
];
```

Lage HTML tabell ved **Array** metode *join()*

- ▶ *join()* returnerer streng ved å sette sammen alle elementene i tabell.
- ▶ Argumentet til *join()* legges inn mellom elementene.

```
const deltagereHTMLTekst = [  
    "<tr><td>9</td><td>Hanne Hannesen</td></tr>",  
    "<tr><td>57</td><td>Gro Grosen</td></tr>"  
];  
const HTMLStreng = deltagereHTMLTekst.join("");
```

- ▶ Lage DOM med element metode *setHTML()*:

```
const tbody = document.querySelector("table").tBodies[0];  
tbody.setHTML(tableText);
```

- Må ikke bruke egenskap eller metode sårbar for XSS angrep.
- Ikke bruk *innerHTML*, *outerHTML*, *insertAdjacentHTML()* eller *setHTMLUnsafe()*.

JavaScript **Array** med HTML tabellrekker

- ▶ DOM manipulasjon er kostbart for nettleser.
 - F.eks. å oppretter tabellrekker på nytt for hver ny deltager.
- ▶ Kan legge selve tabellrekkene i JavaScript tabell.

```
const deltagere = [];  
  
const nydeltager = { startnummer: 127, navn: "Anne Annesen" };  
  
const deltagerrekke = document.createElement("TR");  
deltagere.push(deltagerrekke);  
  
deltagerrekke.insertCell(-1).textContent = nydeltager.startnummer;  
deltagerrekke.insertCell(-1).textContent = nydeltager.navn;
```

- ▶ Legge tabellrekkene inn i HTML tabell.

```
const tbody = document.querySelector("table").tBodies[0];  
tbody.replaceChildren(); // Tømmer innholdet mest effektivt  
  
deltagere.forEach(  
  tr => tbody.appendChild(tr)  
);
```

cloneNode() for å kopiere tabellrekke

- ▶ Å bygge DOM tre fra grunnen av er kostbart for nettleser.
 - Unngå helst mye bruk av **document** sin metode *createElement()*.
- ▶ Bruk av f.eks. *setHTML()* og *innerHTML* er raskere, men:
 - *setHTML()* fungerer kun i noen nyere nettlesere.
 - *innerHTML* gjør applikasjonen sårbar for XSS angrep.
- ▶ En god løsning er å kopiere en eksisterende tabellrekke:

```
const tabellrekke = document.createElement("tr");
const tabellrekke.innerHTML = "<td></td><td></td><td>";

const nydeltager = { startnummer: 127, navn: "Anne Annesen" };
const deltagerrekke = tabellrekke.cloneNode(true);

deltagerrekke.celles[0].textContent = nydeltager.startnummer;
deltagerrekke.celles[1].textContent = nydeltager.navn;
```

- ▶ Løsningen forutsetter at samme tabellrekke objekt gjenbrukes.
 - Kan være et static klasse-felt.